

Classification Presse

Manuel technique

Projet L3 Informatique

Table des matières

1	Objectif du projet	3
2	Architecture générale	3
3	Organisation logique du projet	3
4	Configuration des sources	4
5	Collecte et extraction	4
5.1	Collecte	4
5.2	Extraction	4
5.3	Respect de robots.txt	4
6	Déduplication	5
7	Stockage PostgreSQL	5
7.1	Rôle de la base	5
7.2	Données typiques stockées	5
8	Reconstruction du corpus	5
9	Prétraitement linguistique	6
9.1	Lemmatisation	6
9.2	Stopwords	6
10	Représentation TF-IDF	6
11	K-means	7
12	LDA	7
13	Régression logistique	7
14	Décision INCERTAIN	7
15	Évaluation	8
15.1	Pureté	8
15.2	Adjusted Rand Index	8
15.3	Cohérence LDA	8
15.4	Accuracy	8
15.5	F1-macro	8
15.6	Validation croisée	8

16 Interface web	9
17 Dépendances Python	9
18 Environnement virtuel	9
19 Configuration par .env	9
20 Paquet Debian	10
20.1 Objectif	10
20.2 Construction	10
20.3 Fichier control	10
20.4 Scripts Debian	10
21 Archive des sources	10
22 Licence	10
23 Commandes principales	11
24 Limites techniques	11
25 Évolutions possibles	11
26 Résumé technique	12

1 Objectif du projet

Classification Presse est un projet de traitement automatique du langage et de fouille de données appliqué à des articles de presse francophones.

L'objectif est de construire un corpus thématique, de préparer les textes, de les représenter sous forme numérique, d'appliquer plusieurs méthodes de fouille de données puis de proposer une classification pour un nouvel article.

Le logiciel couvre trois dimensions principales :

- ingénierie des langues : constitution et préparation du corpus ;
- fouille de données : représentation, clustering, topics, classification et évaluation ;
- développement de logiciel libre : distribution sous forme de paquet Debian, interface utilisateur et fourniture du code source.

2 Architecture générale

Le fonctionnement peut être résumé par le pipeline suivant :

```

Sources web
|
v
Collecte des URLs et des articles
|
v
Extraction du titre, du texte et des metadonnees
|
v
PostgreSQL
|
v
Nettoyage et lemmatisation
|
+--> TF-IDF -----> K-means
| \-> Regression logistique
|
+--> Bag-of-words --> LDA
|
v
Evaluation, comparaison et interface web locale

```

L'interface n'implémente pas directement les algorithmes. Elle sert de couche utilisateur et appelle les fonctions du pipeline.

3 Organisation logique du projet

L'organisation exacte peut évoluer légèrement, mais les composants principaux sont :

config/sources.yaml définition des sources, thèmes et limites ;

sql/01_schema.sql création de la structure PostgreSQL ;

src/news_classifier/ code Python principal ;

compare_models.py comparaison des résultats des trois méthodes ;

visualizations.py génération des graphiques de résultats et de comparaison ;

outputs/ résultats générés par les analyses ;

packaging/debian/ fichiers nécessaires au paquet Debian ;
docs/ documentation utilisateur et technique ;
build_deb.sh construction du paquet **.deb** et de l'archive source ;
requirements.txt dépendances Python.

4 Configuration des sources

Le fichier :

config/sources.yaml

contient les sources du corpus.

Une source possède principalement :

- un type ;
- une URL ;
- un thème ;
- une limite d'articles.

La configuration actuelle contient six catégories et vise dix articles par thème.

Le logiciel utilise les rubriques des médias comme **labels de référence** pour l'évaluation. Il est préférable de parler de labels de référence plutôt que de gold standard strict, puisque ces catégories proviennent de rubriques éditoriales.

5 Collecte et extraction

5.1 Collecte

Le programme lit **sources.yaml**, visite les pages de départ et découvre des liens d'articles compatibles avec la source.

Pour chaque source, il tente de collecter jusqu'au nombre demandé d'articles.

5.2 Extraction

L'extraction utilise des requêtes HTTP et une analyse HTML avec BeautifulSoup et **lxml**.

Les informations récupérées peuvent comprendre :

- URL ;
- titre ;
- texte principal ;
- auteur, lorsqu'il est disponible ;
- date de publication, lorsqu'elle est disponible ;
- nom du site ;
- thème de référence.

Un article doit contenir suffisamment de texte pour être retenu. Dans la version actuelle, un minimum de 80 mots est utilisé pour éviter les pages de navigation ou les pages presque vides.

5.3 Respect de robots.txt

Le collecteur vérifie les règles de **robots.txt** avant de récupérer certaines ressources. Cela permet de tenir compte des règles d'accès automatisé publiées par le site.

6 Déduplication

Le projet utilise un checksum du contenu afin d'éviter d'enregistrer plusieurs fois le même article.

Une empreinte telle que SHA-256 produit une valeur dépendant du contenu.

Si deux documents ont la même empreinte, ils peuvent être considérés comme des doublons avec une forte probabilité.

7 Stockage PostgreSQL

7.1 Rôle de la base

PostgreSQL sert à conserver les articles et leurs métadonnées de manière structurée.

Le corpus n'est donc pas seulement présent dans la mémoire du programme.

La base facilite :

- la conservation du corpus ;
- les requêtes par thème ;
- la détection des doublons ;
- l'export des données ;
- la reconstruction des analyses.

7.2 Données typiques stockées

Selon le schéma du projet, un enregistrement peut comprendre :

- identifiant ;
- URL ;
- titre ;
- texte ;
- thème ;
- site ;
- checksum ;
- date d'import ;
- encodage ;
- format source.

8 Reconstruction du corpus

Dans la version actuelle, l'action **Construire le corpus** produit un corpus reproductible en termes de taille et d'équilibre :

<code>6 themes x 10 articles = 60 articles</code>

Le processus commence par supprimer le corpus précédent puis recommence la collecte.

Cela évite l'accumulation d'articles entre plusieurs exécutions.

L'objectif est d'avoir :

Thème	Nombre
Tech	10
Sciences	10
Sport	10
Politique	10
Économie	10
Culture	10

9 Prétraitement linguistique

Le prétraitement prépare les textes avant les algorithmes de fouille de données.

Les principales opérations sont :

- normalisation ;
- suppression du bruit ;
- tokenisation ;
- lemmatisation ;
- suppression des mots vides ;
- filtrage de certains éléments non informatifs.

9.1 Lemmatisation

La lemmatisation est réalisée avec spaCy et le modèle français `fr_core_news_sm`.

Exemple :

mangeait, mangerons, mange -> manger

Cette opération réduit plusieurs formes grammaticales vers une forme commune.

9.2 Stopwords

Des mots très fréquents comme *le*, *de*, *et* ou *un* apportent souvent peu d'information pour distinguer les thèmes.

Ils sont donc généralement supprimés.

10 Représentation TF-IDF

Les algorithmes de machine learning ne travaillent pas directement avec les phrases. Les textes sont donc convertis en vecteurs numériques.

TF-IDF attribue un poids élevé à un terme lorsqu'il est important dans un document mais moins fréquent dans le reste du corpus.

Cette représentation produit une matrice :

documents x termes

Chaque article devient alors un vecteur numérique.

11 K-means

K-means est utilisé pour le clustering non supervisé.

Le nombre de clusters est fixé à six afin de pouvoir comparer les groupes obtenus aux six thèmes du corpus.

Le fonctionnement général est :

1. initialisation de centres de clusters ;
2. affectation des articles au centre le plus proche ;
3. recalcul des centres ;
4. répétition jusqu'à stabilisation.

Les labels de référence ne sont pas utilisés pour créer les clusters.

Ils sont utilisés ensuite pour l'évaluation et pour associer chaque cluster à un thème de référence. Cette association permet également d'utiliser K-means lorsqu'une nouvelle URL est testée : le nouvel article est comparé aux centroïdes et le cluster le plus proche fournit une proposition de thème.

12 LDA

LDA, pour Latent Dirichlet Allocation, est une méthode de modélisation thématique.

Elle considère qu'un document peut être composé de plusieurs thèmes latents et qu'un thème est caractérisé par une distribution de mots.

Le projet utilise LDA pour extraire des thèmes et observer si ces thèmes se rapprochent des catégories éditoriales connues.

Les tokens prétraités sont convertis en représentation de type *bag-of-words* avec Gensim avant l'application de LDA.

Après l'analyse du corpus, les topics sont associés aux thèmes de référence. Cette association permet aussi d'appliquer LDA à une nouvelle URL et de proposer un thème à partir du topic dominant.

La cohérence `c_v` est utilisée pour mesurer la cohérence sémantique des mots d'un topic.

13 Régression logistique

La régression logistique est le modèle supervisé utilisé pour prédire directement l'un des six thèmes connus pour une nouvelle URL.

Contrairement à K-means et LDA, ce modèle apprend à partir des labels connus. Les trois méthodes sont néanmoins appliquées au même nouvel article afin de confronter leurs résultats.

La chaîne est :

```
article -> pretraitement -> TF-IDF -> regression logistique -> probabilites
```

Le modèle produit une probabilité pour chacun des six thèmes.

14 Décision INCERTAIN

Le logiciel ne force pas systématiquement une catégorie.

Il peut retourner :

```
INCERTAIN
```

si la confiance est trop faible.

La logique actuelle utilise deux critères :

- le meilleur score doit être suffisamment élevé ;
- l'écart entre les deux meilleurs thèmes doit être suffisamment important.

Dans la version actuelle, la décision devient incertaine si :

```
meilleure probabilite < 0.35  
OU  
ecart entre les deux premiers < 0.08
```

Cette règle évite de présenter une prédiction comme certaine lorsque le modèle hésite réellement entre plusieurs thèmes.

15 Évaluation

15.1 Pureté

La pureté mesure, dans chaque cluster, la proportion d'éléments appartenant au thème de référence majoritaire.

Une valeur proche de 1 indique des clusters relativement homogènes.

15.2 Adjusted Rand Index

L'ARI compare deux partitions en corrigeant une partie de l'accord attendu par hasard.

Interprétation générale :

- 1 : accord parfait ;
- autour de 0 : accord proche du hasard ;
- valeur négative : accord inférieur à celui attendu aléatoirement.

15.3 Cohérence LDA

La cohérence mesure si les mots associés à un topic ont tendance à former un ensemble sémantiquement cohérent.

Ce n'est pas une mesure d'accuracy.

15.4 Accuracy

L'accuracy mesure la proportion totale de prédictions correctes.

15.5 F1-macro

Le F1-macro calcule un score F1 séparément pour chaque classe puis effectue une moyenne donnant le même poids à chaque thème.

15.6 Validation croisée

La régression logistique est évaluée par validation croisée à cinq plis.

Le corpus est séparé plusieurs fois en parties d'entraînement et de validation afin d'éviter d'évaluer le modèle uniquement sur les données qui ont servi à l'apprendre.

16 Interface web

L'interface est développée avec Flask.

Elle est exécutée localement et permet d'utiliser le logiciel sans lancer manuellement les différents scripts Python.

Elle expose trois fonctions :

1. construire le corpus ;
2. lancer l'analyse ;
3. classer une URL avec K-means, LDA et la régression logistique.

L'interface affiche également les statistiques du corpus, les résultats d'évaluation, les graphiques des modèles et les graphiques de comparaison produits pour une nouvelle URL.

17 Dépendances Python

Les principales bibliothèques utilisées sont :

- `requests` : accès HTTP ;
- `beautifulsoup4` et `lxml` : analyse HTML ;
- `psycopg2-binary` : PostgreSQL ;
- `pandas` et `numpy` : manipulation de données ;
- `scikit-learn` : TF-IDF, K-means, régression logistique et métriques ;
- `spacy` : traitement linguistique ;
- `NLTK` : ressources linguistiques complémentaires ;
- `gensim` : LDA et cohérence ;
- `matplotlib` : génération des visualisations ;
- `joblib` : sérialisation de modèles ;
- `PyYAML` : lecture de la configuration ;
- `Flask` : interface web.

18 Environnement virtuel

Les dépendances Python sont installées dans un environnement virtuel propre au logiciel afin de limiter les conflits avec les bibliothèques du système.

L'espace utilisateur principal est :

```
~/local/share/classification-presse
```

19 Configuration par `.env`

Le projet peut utiliser un fichier `.env` pour stocker certains paramètres de configuration, par exemple :

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/presse
USER_AGENT=ProjetL3ClassificationPresse/1.0
```

Le fichier `.env.example` sert de modèle et peut être distribué avec les sources.

Le fichier `.env` réel peut contenir des paramètres propres à la machine et n'a pas vocation à être publié systématiquement.

20 Paquet Debian

20.1 Objectif

Le paquet Debian transforme le projet en logiciel installable.

Le fichier produit est :

```
classification-presse_1.0.0-1_all.deb
```

20.2 Construction

La construction est lancée avec :

```
./build_deb.sh
```

Le script prépare l'arborescence attendue par Debian, copie les fichiers et appelle notamment :

```
dpkg-deb
```

20.3 Fichier control

Le fichier `packaging/debian/control` contient les métadonnées du paquet :

- nom ;
- version ;
- architecture ;
- dépendances ;
- description.

20.4 Scripts Debian

Le paquet peut inclure des scripts d'installation tels que `postinst` ou `postrm`.

Ils permettent d'effectuer certaines opérations après installation ou suppression.

21 Archive des sources

La construction produit également une archive :

```
classification-presse_1.0.0-1_source.tar.gz
```

Elle contient le code source du projet afin de permettre son étude, sa modification et sa reconstruction.

Cette archive est distincte du paquet binaire `.deb`.

22 Licence

Le projet inclut un fichier `LICENSE`.

La licence libre précise les droits accordés aux utilisateurs concernant l'utilisation, la copie et la modification du code source.

23 Commandes principales

Installation :

```
sudo apt install ./classification-presse_1.0.0-1_all.deb
```

Lancement :

```
classification-presse
```

Configuration PostgreSQL en cas de besoin :

```
sudo classification-presse-db-setup
```

Vérification de l'installation :

```
dpkg -l | grep classification-presse  
command -v classification-presse
```

Désinstallation :

```
sudo apt remove classification-presse
```

24 Limites techniques

Les principales limites actuelles sont :

- corpus de petite taille ;
- seulement deux familles de médias pour six thèmes ;
- risque de corrélation entre thème et style du média ;
- extracteur HTML non universel ;
- dépendance à la disponibilité des sites externes ;
- catégories parfois ambiguës ;
- performances de validation croisée ne garantissant pas la même précision sur des sites externes.

25 Évolutions possibles

Des améliorations possibles seraient :

- augmenter le nombre d'articles ;
- utiliser plusieurs médias pour chaque thème ;
- améliorer l'extraction des pages dynamiques ;
- comparer d'autres classifieurs ;
- ajouter une vraie séparation apprentissage / validation / test sur un corpus plus important ;
- enrichir la documentation et les tests automatisés ;
- tester le paquet sur plusieurs distributions Debian/Ubuntu.

26 Résumé technique

Le projet suit la chaîne suivante :

Configuration YAML

```
-> collecte web
-> extraction
-> PostgreSQL
-> nettoyage / lemmatisation
-> TF-IDF pour K-means et la regression logistique
-> bag-of-words pour LDA
-> K-means / LDA / regression logistique
-> evaluation et comparaison
-> visualisations
-> interface Flask
-> paquet Debian
```

Cette architecture permet de relier la constitution d'un corpus linguistique, l'analyse de données textuelles et la transformation du projet en logiciel déployable.